

PRE-CLASS READING 1 OF 5

Workflow or agent? — Anthropic on building effective agents

Anthropic (2024). “Building effective agents.” *Engineering blog*, 19 December 2024 — plus the 2025–26 follow-up posts that update it. This is the single most-cited practitioner text on when to build an agent and when not to.

ASSIGNED READING

Read “Building effective agents” end to end, including both appendices (≈ 20 min). The **Note on currency** below lists four follow-up posts that update it — skim the two marked as worth skimming, and simply note that the other two exist.

» [source](#) ~20 min • [this summary + reasoning notes](#) ~24 min

Note on currency. The post is from December 2024 and its core distinction has held up, but the field moved underneath it. Anthropic has published four follow-ups since. **Worth skimming:** “Effective context engineering for AI agents” (29 Sep 2025) and “Demystifying evals for AI agents” (9 Jan 2026) — between them they carry most of what changed. **Worth knowing exist:** “Writing effective tools for agents” (11 Sep 2025) and “Effective harnesses for long-running agents” (26 Nov 2025); both are picked up in Capsule 2. Where any of the four change or sharpen the original advice, this brief says so. Everything attributed to Anthropic below is from Anthropic’s own engineering posts, not from third-party commentary.

KEY TAKEAWAY

Most things people call “agents” should not be agents. Anthropic splits **agentic systems** into two kinds: **workflows**, where “LLMs and tools are orchestrated through predefined code paths”, and **agents**, where “LLMs dynamically direct their own processes and tool usage.” Workflows are predictable and cheap; agents are flexible and expensive. Their standing advice is to find “the simplest solution possible, and only increas[e] complexity when needed” — because “agentic systems often trade latency and cost for better task performance.” For a large share of real tasks, one well-retrieved, well-exampled model call is enough. The engineering skill this course is after is not *building the biggest agent* — it is **knowing which rung of that ladder your problem actually sits on**, and being able to defend the choice on cost, latency and failure risk.

WHY IT MATTERS FOR THIS COURSE

Week 3 taught you to steer a **single call** — prompt it, structure its output, and measure it against an eval set. Week 4 is the step up: **orchestrating a loop**. This reading gives you the vocabulary for the whole of Class 4 — augmented model, workflow patterns, the agent loop, the agent-computer interface — and, more importantly, it gives you the *restraint*. Capsule 3 asks whether an agent is worth its token bill; you cannot answer that without first being able to name the cheaper thing you could have built instead. It also sets up the case: Anthropic is both the author of this restraint doctrine **and** a company that has to decide commercially how far to push autonomy.

Feeds Class 4: C1 (what an agent is • the chooser and the agent patterns • agent vs prompt vs RAG) • C2 (tool design • failure modes • agent evals) • C3 (when the loop is not worth the tokens • the Anthropic case).

MAIN POINTS

The building block: the augmented model

- **Start from one call, not from a framework.** The base unit is a model call with three add-ons bolted on: **retrieval** (it can look things up), **tools** (it can do things) and **memory** (it can keep things). Modern models “actively use these capabilities — generating their own search queries, selecting appropriate tools, and determining what information to retain.” Call this the **augmented-model layer**. Nothing here loops, and nothing here is orchestrated — a single call has no control flow above it. Anthropic’s advice is to get this layer right — clear tool definitions, a tailored interface — before you put any control flow on top, whether that is a fixed two-step chain or a full agent loop.
- **Why the augmented-model layer comes first.** Almost every agent failure you will meet in Capsule 2 is really a failure of **this layer** — the model call and its tools, retrieval and memory — surfacing two steps later: a vague tool description, a tool that returns 4,000 tokens of noise, a retrieval step that fetched the wrong document. Orchestration on top of a bad augmented model does not fix it — it runs the same mistake five times and bills you for each.

Five workflow patterns — the cheap rungs of the ladder

- **1 • Prompt chaining.** Fixed sequence: call 1’s output is call 2’s input, with programmatic **gates** in between that check the intermediate result before you spend the next call. *Worked example:* draft marketing copy → gate: “is it under 60 words and does it name the product?” → if yes, translate to Mandarin. Use when the task genuinely decomposes into fixed subtasks and you are buying accuracy with latency.
- **2 • Routing.** Classify the input first, then send it down a specialised path with its own prompt. *Worked example:* a support inbox where refund requests, technical faults and general questions get three different prompts —

and, separately, easy questions get a small cheap model while hard ones get a frontier model. Routing is the pattern most often left on the table: it is the single easiest way to cut cost without cutting quality.

- **3 • Parallelisation.** Two flavours. **Sectioning** splits a task into independent pieces run at the same time (answer the user *and* run a safety check on the same input). **Voting** runs the *same* task several times and compares — Week 3’s self-consistency idea, reused. *Worked example:* three differently-prompted reviewers each scan a code diff for vulnerabilities; you keep anything two of them flag.
- **4 • Orchestrator-workers.** A lead model **decides at runtime** how to split the task, hands pieces to workers, and synthesises the results. The distinction from parallelisation is exactly this: “subtasks aren’t pre-defined, but determined by the orchestrator based on the specific input.” *Worked example:* a change that might touch three files or eleven — you cannot know until you look.
- **5 • Evaluator-optimiser.** One model produces, a second critiques against explicit criteria, the first revises; loop until the critic passes it. *Worked example:* literary translation, where “more idiomatic” is real but not machine-checkable. Only worth it when you can articulate the criteria **and** a second pass measurably improves things — otherwise you have doubled the bill to feel thorough.
- **Read the five as a cost ladder, not a menu.** Roughly: chaining and routing add a call or two and stay predictable; parallelisation multiplies calls but not wall-clock time; orchestrator-workers multiplies calls *and* becomes unpredictable in advance; evaluator-optimiser multiplies calls by an unknown number of rounds. Your token bill and your ability to forecast it fall as you go down the list.

The agent proper — and the one thing that makes it work

- **What an agent is.** After the task is clarified, the agent “plan[s] and operate[s] independently”, deciding its own next step and returning to the human only for information or judgement. There is no predefined path.
- **The non-negotiable condition: ground truth at every step.** An agent must “gain ‘ground truth’ from the environment at each step (such as tool call results or code execution) to assess its progress.” This is the line to remember. An agent that cannot check its own work against the world is not an agent — it is a monologue with tool-shaped noises in it. It is also why coding is the strongest agent domain in the post: tests run, and either pass or fail. *Diagnostic to carry into Capsule 2:* before you build, ask *what will tell this loop it is wrong, and how fast?* If the honest answer is “a human, next week”, do not build an agent.
- **When to use one.** “Open-ended problems where it’s difficult or impossible to predict the required number of steps, and where you can’t hardcode a fixed path” — plus enough trust to let it act, sandbox testing, and guardrails, because of “higher costs and potential for compounding errors.” Compounding is the key word: a 95%-reliable step is 95% reliable; twenty of them in a row is about 36%.

Three principles, and the interface that carries them

Anthropic states three principles: keep the design simple, make the agent’s planning visible, and craft the interface the model sees. Quoted phrases are theirs. Anything after Our reading is this brief’s interpretation, not the post.

- **1 • Simplicity.** “Maintain simplicity in your agent’s design.” **Our reading:** every tool you add enters the menu the model chooses from, so more tools mean more chances to choose wrong — and each tool’s definition is re-sent, and re-billed, on every turn. Extra components also widen a failure surface you cannot test exhaustively, because the paths are not enumerable. The trap to avoid: a framework that hides the prompt and the loop, so that months later nobody can explain what the agent did, because nobody has seen the text actually sent to the model.
- **2 • Transparency.** “Prioritize transparency by explicitly showing the agent’s planning steps.” **Our reading:** this serves two audiences. For the operator, a visible plan is the only moment to intervene before something irreversible happens — so “show the plan, then execute” is a governance control, not a presentation choice. For the developer, the trajectory is the only artefact that exists: hide it and you cannot tell whether the run failed at step 2 or step 7. Logs and audit trails are this principle made durable.
- **3 • A crafted agent-computer interface (ACI).** Anthropic argues the interface the model sees deserves the effort normally spent on a human interface (HCI): include example usage, edge cases, unambiguous parameters and clear boundaries. **Our reading:** the tool description is the entire manual the model gets. It cannot ask a colleague what a parameter means, hover a tooltip, or try the tool in staging. Whatever you wrote is all it knows — which is why the next three rules are worth memorising.
- **Interface rule 1 — room to think.** Give the model “enough tokens to ‘think’ before it writes itself into a corner.” **Our reading:** generation runs left to right and a token, once emitted, cannot be retracted. A schema that demands the answer as its first field forces the model to commit before it reasons. Put a reasoning field first, or allow a line of thought before the structured output.
- **Interface rule 2 — familiar formats.** Keep formats “close to what the model has seen naturally occurring in text.” **Our reading:** a training-distribution argument. Markdown, ordinary code and plain JSON are everywhere in the training data; heavily escaped strings and bespoke delimiters are not. An unfamiliar format is not merely awkward for the model, it is statistically rare.

- **Interface rule 3 — no bookkeeping.** Remove overhead such as counting characters, escaping strings or tracking indentation. **Our reading:** clerical work spends model effort that should go on the task, and one miscount corrupts the whole output.
- **Interface rule 4 — poka-yoke the tool.** Mistake-proof the interface rather than instructing the model not to err. Anthropic's case: on their SWE-bench agent, requiring **absolute** filepaths eliminated relative-path errors outright — not reduced, eliminated. **Our examples:** an enum instead of free text for a status; require `patient_id` rather than `patient_name`; default `dry_run` to true; split `draft_email` from `send_email` so sending takes a deliberate second call.
- **Not in the post — why an interface beats an instruction.** A sentence in the prompt is paid for on every call, competes for attention with everything else in the window, and stops working the day you change model. A constraint in the tool signature is paid for once and holds whichever model you plug in. Prompts persuade; interfaces constrain.
- **Not in the post — a caveat worth adding.** A model's stated reasoning is not a guaranteed account of its actual computation. Anthropic's later research ("**Reasoning Models Don't Always Say What They Think**", arXiv:2505.05410, May 2025) found reasoning models often omit the influences that actually changed their answer. A visible plan is an excellent debugging surface and a poor proof of correctness.

What the 2025-26 posts add

- **Context engineering (Sep 2025).** The unit of design is no longer the prompt but the whole context window — instructions, tools, retrieved data, history. Anthropic names "**context rot**": quality degrades as the window fills, because attention is a finite budget and a transformer relates every token to every other. The rule is to find "the smallest possible set of high-signal tokens" that gets the outcome. Long-horizon tactics: **compaction** (summarise the history when the window fills), **structured note-taking** (the agent writes to an external file it can re-read), and **sub-agents** that explore and return a condensed ~1,000-2,000-token summary rather than their whole trace.
- **Agent evals (Jan 2026).** Agent evaluation differs from Week 3's prompt evaluation because behaviour is multi-turn, stateful and non-deterministic. Their guidance: start with **20-50 tasks drawn from real failures**; use code-based graders for what is deterministic, model graders with calibrated rubrics for what is not, humans for spot checks; and "**grade what the agent produced, not the path**", so a valid-but-unexpected route is not marked wrong. Their diagnostic is a good one: with a frontier model, "a 0% pass rate across many trials ... is most often a signal of a broken task, not an incapable agent." This is Week 3's ten-case harness, grown up — and it is the discipline A2 will expect.
- **Harnesses (Nov 2025).** For work that outlives one context window, the scaffolding *around* the model does the heavy lifting: an initialiser session that sets up a feature list and a progress file, then sessions that read the git log and the progress notes first, do one feature, test end-to-end, commit, and write down what they did. Failure modes are mostly bookkeeping failures — declaring victory early, leaving undocumented bugs, marking things done without testing.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to give you a direction to push on; the fuller reasoning is in the section after.

- Take one task you actually do at work that someone has told you "an AI agent could do." Walk it down Anthropic's ladder — single call, chain, router, agent — and stop at the first rung that would genuinely do the job. Which rung, and what specifically is on the rung above that you do not need?

Clue. *Diagnose before you prescribe. Ask two questions of the task: can you write down the steps in advance, and does the number of steps vary with the input? If both answers are "yes, fixed", it is a workflow, no matter how sophisticated it feels. Most people land one or two rungs lower than they expected, and the interesting part of your answer is what you had to give up to stay there.*

- Anthropic says an agent must get "ground truth" from its environment at every step. For your task from Q1 — what is the ground truth, how quickly does it arrive, and what happens to the loop if it never arrives?

Clue. *Start from what the environment can actually tell you, not from what you wish it could. Sort candidate signals by latency and objectivity: a failing unit test is instant and objective; a database row that either exists or does not is instant and objective; a customer's satisfaction is neither. Then ask what the loop does in the gap — if nothing corrects it for three steps, you are paying for three steps of confident drift.*

- A 95%-reliable step, run twenty times in a row, gives you roughly a 36% chance of a clean run. Given that arithmetic, why does anyone build long agent loops at all — and what would you change so the number is not fatal?

Clue. *Do the arithmetic first ($0.95^{20} \approx 0.358$), then attack it from three sides rather than one: raise the per-step reliability, cut the number of steps, or make failures **recoverable** so a step failing is not a run failing. Anthropic's*

checkpoints, gates and verification loops are all the third kind. Notice which of the three your own answer reached for — most people reach for the first, which is the hardest to move.

- You keep seeing the same tool error in your agent's trace. Compare two fixes: adding a sentence to the system prompt, versus changing the tool's signature so the error cannot be expressed. When is each right, and what does the choice cost?

Clue. This is the poka-yoke question, and it is not a trick — the prompt fix is sometimes right. Compare on three axes: does it eliminate the error class or only make it less likely; who pays (a prompt sentence is paid for on every single call, forever, in tokens and attention budget); and what breaks when the model is swapped for a different one next quarter. Anthropic's absolute-filepath change is the cleanest possible example of one side of this.

- Week 3 gave you a ten-case eval harness for one prompt. Anthropic suggests 20–50 tasks for an agent, graded on outcome rather than path. What breaks if you simply reuse your Week 3 harness on an agent — and what would you have to add?

Clue. Enumerate what changed between the two objects being tested: one call became many; the output became a trajectory plus a side-effect on the world; the same input can now produce different runs. Each of those breaks a specific assumption in a single-turn harness — find the assumption before you design the fix. And keep hold of the trap in the other direction: grading the path is tempting precisely because it is easy to automate.

HOW TO THINK ABOUT THESE — read after you have had a go

These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so they are written to stand on their own.

Q1 — Which rung, and what you are not buying

The ladder is really a question about **who decides the control flow**. In a single call, you decide everything. In a chain, you decide the order and the model fills the slots. In a router, you let the model decide *one* thing — which path. In an agent, you hand over the whole sequence. Each handover buys flexibility and pays in three currencies: tokens, latency, and your ability to predict what will happen before it happens. The last one is the one people forget to price. A workflow can be tested exhaustively because its paths are enumerable; an agent cannot, which is why the eval discussion arrives immediately after. Most real business tasks — invoice triage, first-line support, report drafting — are chains or routers with a bad reputation. The honest test is whether the *number of steps varies with the input*. If it does not, you are looking at a workflow, and building an agent for it means paying an autonomy premium for flexibility you will never use.

Q2 — Ground truth is the thing that makes the loop converge

A loop without feedback is not a loop, it is a recursion with a token meter attached. What makes coding agents unusually strong is that the environment answers immediately and objectively: the tests pass or they do not, the build compiles or it does not. Compare a “market research agent” — its output is plausible prose, and nothing in its environment ever says *no*. So it never self-corrects, and because it never self-corrects, more steps make it more confident rather than more right. When you sort your candidate signals by latency and objectivity, you usually find one of three situations. There is a fast objective signal — build the agent, and wire the signal in as the first thing you do. There is a slow or subjective signal — build a workflow with a human gate at the point where the signal would have been. Or there is no signal at all — in which case the interesting realisation is that you do not have an AI problem, you have an unspecified problem, and no amount of autonomy will fix an objective nobody can state.

Q3 — Compounding, and the three ways out

The arithmetic is unforgiving and worth having at your fingertips: $0.95^{20} \approx 0.358$, $0.99^{20} \approx 0.818$. You can attack it three ways. **Raise reliability per step** — real, but you are fighting for fractions of a percent at the frontier, and it is the lever you control least. **Cut the steps** — usually the biggest single win, and it is what routing and good tool design actually do: a tool that returns the right answer in one call replaces four exploratory ones. **Make failure recoverable** — the underrated one. If a failed step is caught by a gate, retried, or rolled back to a checkpoint, then step reliability stops multiplying and starts merely costing you time and tokens. This is precisely what Anthropic's harness advice does with commits and progress files: it converts a fatal run-level failure into a survivable step-level one. Notice too that the arithmetic explains the whole restraint doctrine. It is not that agents are bad; it is that autonomy without recovery scales badly, and every serious agent design is mostly recovery machinery.

Q4 — Prompt fix versus interface fix

The distinction is between **suppressing a behaviour** and **removing its possibility**. A prompt sentence lowers the probability of the error and does so on every call — you pay tokens for it forever, you dilute attention on everything else in the window, and it is fragile the moment you change models, because it depends on this model attending to that sentence. Changing the tool signature so relative paths simply cannot be passed removes the error class permanently and costs nothing per call. So the default is: **if you can make the mistake unrepresentable, do that**. The prompt fix earns its place in two cases — when the “error” is actually a judgement call rather than a malformed input (you cannot type-check “be conservative when the customer is angry”), and when you are still learning what

the real failure distribution is and want a cheap reversible patch while you watch. The deeper point transfers well beyond agents: the model is one component in a system, and system design gives you a place to put the guarantee. Prompts persuade; interfaces constrain. Prefer the constraint.

Q5 — Why your Week 3 harness does not survive contact with an agent

Three assumptions break at once. First, **one call became many**: your harness compared an output to an expectation, but an agent produces a trajectory — a sequence of tool calls, observations and revisions — and the failure you care about is often three steps upstream of the bad output. Second, **the output includes side-effects**: the agent may have written a file, sent an email, or spent money, none of which your string comparison inspects, and one of which you cannot undo. Third, **the same input can produce different runs**, so a single pass tells you nothing; you need multiple trials and to think in pass rates rather than pass/fail. What you add: more and more realistic tasks (20–50 drawn from failures you actually saw, not invented ones); a mixture of graders, with code-based checks for what is deterministic and a rubric-scored model judge for what is not; isolation between trials so one run cannot contaminate the next; and negative cases, because a set of only-positive tasks optimises an agent into an eager yes-machine. Keep the discipline of grading the *outcome*: the moment you grade the path, you have quietly redefined “correct” as “did what I imagined”, and you will penalise the better solution you did not think of.

APPENDIX — THE VOCABULARY, EXPLAINED

*The one distinction that carries the whole reading: **who decides the route through the system, and when** — decided in advance, by you, in code; or decided in the moment, by the model. Everything below is a variation on that.*

Agentic system

- **Plain.** Any system where a model does more than answer once — it looks things up, uses tools, or takes several turns.
Technically. Anthropic’s umbrella term covering both workflows and agents; the distinction between them is architectural, not a matter of capability.

Workflow

- **Plain.** You write the steps; the model fills them in. The route through the system is decided before anything runs.
Technically. “Systems where LLMs and tools are orchestrated through **predefined code paths**.” Testable exhaustively, because the paths are enumerable in advance.

Agent

- **Plain.** You state the goal; the model decides the steps as it goes, and stops when it thinks it is done.
Technically. “Systems where LLMs **dynamically direct their own processes and tool usage**.” Requires ground truth from the environment at each step, plus a stopping condition and a step cap.

Ground truth

- **Plain.** Something outside the model that can tell it that it is wrong — and tell it quickly.
Technically. Anthropic’s condition for an agent: it must “gain ‘ground truth’ from the environment at each step (such as tool call results or code execution)”. Judge a candidate signal on two axes — how fast it arrives, and whether it is settled by a test or by an opinion. A failing unit test is instant and objective; a customer’s satisfaction is neither. Note what it is not: ground truth is not a correct answer, it is a signal that can contradict the model. Where none exists, more steps make an agent more confident rather than more right.

Orchestration

- **Plain.** The part that decides what runs, in what order, and what happens when something fails.
Technically. Any control flow above a single model call. A fixed chain or a router is orchestration — you decide the path in code. An agent loop is orchestration — the model decides the path at runtime. Orchestration also owns the parts nobody demos: retries, gates, sub-agents and the stop condition. A single call has no orchestration; every rung above it does.

Augmented LLM

- **Plain.** One model call with a search box, a toolbox and a notebook attached.
Technically. Model + retrieval + tools + memory. The base unit of every pattern above; Anthropic advises tailoring it and getting the interfaces clean before adding any orchestration.

Agent-computer interface (ACI)

- **Plain.** The tool descriptions, argument names and output formats the model sees — its user interface onto your systems.
Technically. Deserves the design effort normally given to an HCI (human-computer interface): example usage, edge cases, unambiguous parameters, formats close to naturally occurring text, and enough room for the model to “think” before committing.

Poka-yoke

- **Plain.** Mistake-proofing: change the tool so the wrong thing cannot be done, instead of asking nicely.
Technically. Toyota Production System term. Anthropic's worked case: requiring absolute filepaths in a file-edit tool eliminated relative-path errors on their SWE-bench agent entirely.

Compounding error

- **Plain.** Small per-step failure rates multiply into a large per-run failure rate.
Technically. For n independent steps at reliability r , run reliability is r^n : $0.95^{20} \approx 0.358$. Mitigated by fewer steps, higher per-step reliability, or — most practically — recovery machinery (gates, checkpoints, retries) that stops step failures from being run failures.

Context rot

- **Plain.** The fuller the context window, the worse the model gets at using any one thing in it.
Technically. Anthropic's term (Sep 2025) for degradation as context grows; grounded in the transformer's n^2 token-to-token attention and a finite "attention budget". Countered by compaction, external notes, and sub-agents returning ~1,000–2,000-token summaries.

Harness

- **Plain.** The scaffolding around the model that lets work continue across sessions: setup scripts, a to-do list, a progress file, commits.
Technically. Anthropic (Nov 2025): an initialiser session builds the feature list and progress artefacts; each later session reads git log and progress notes first, implements one feature, verifies end-to-end, commits, and records state. Most long-agent failures are bookkeeping failures, not reasoning failures.

Outcome grading vs path grading

- **Plain.** Judge what the agent produced, not the route it took to get there.
Technically. Anthropic (Jan 2026): path-based graders are brittle and penalise valid alternative solutions. Start with 20–50 tasks drawn from real failures; mix code-based, model-based and human graders; isolate trials; include negative cases.

Restraint

- **Plain.** Build the simplest thing that does the job, and add autonomy only where the simpler rung genuinely fails.
Technically. Anthropic's stated principle: find "the simplest solution possible, and only increas[e] complexity when needed", because "agentic systems often trade latency and cost for better task performance." In this course restraint is a design discipline you must be able to defend on cost, latency and failure risk — not modesty, and not a reason never to build an agent.

SOURCE LINKS

Building effective agents (the assigned reading) • www.anthropic.com/engineering/building-effective-agents

Effective context engineering for AI agents • www.anthropic.com/engineering/effective-context-engineering-for-ai-agents (29 Sep 2025)

Writing effective tools for agents — with agents • www.anthropic.com/engineering/writing-tools-for-agents (11 Sep 2025)

Effective harnesses for long-running agents • www.anthropic.com/engineering/effective-harnesses-for-long-running-agents (26 Nov 2025)

Demystifying evals for AI agents • www.anthropic.com/engineering/demystifying-evals-for-ai-agents (9 Jan 2026)

Companion • **Reading 2** (ReAct) is the mechanism inside the loop this reading describes from the outside; **Reading 5** (Devin) is what happens when the restraint in this reading is not applied.

CAVEATS

1. This summary is not a substitute for the post. It is a map, not the territory — the original is short, well-written and free; read it.
2. The December 2024 post predates the tool-use, context-management and eval practice that grew up around it during 2025–26. Treat the workflow/agent distinction as durable and the surrounding engineering detail as a moving target.
3. Anthropic builds and sells agent infrastructure. The restraint advice is well-argued and widely echoed, but it is not disinterested — a point worth holding on to when we discuss the case in Capsule 3.
4. This summary was generated and fact-checked using Anthropic models. Quoted phrases are from Anthropic's engineering posts; the worked examples, the cost-ladder framing and the compounding arithmetic are the instructor's, not Anthropic's. Flag anything that looks wrong.